



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: CI/CD, Pipelines e Testes Automatizados

Curso: ENGENHARIA DE INTELIGÊNCIA ARTIFICIAL E MLOPS

Professor: Wander Maia da Silva

Instituição: IEC PUC Minas

Laboratórios Práticos — Unidade 1

Pré-requisito: conta no Azure DevOps criada conforme o [material-apoio-contas-cicd](#).

Índice

- [Lab 1 — Primeiro pipeline no Azure DevOps](#)
- [Lab 2 — Agente auto-hospedado e estratégias de deployment](#)
- [Lab 3 — GitOps com Azure DevOps e repositório de manifests](#)
- [Lab 4 — Boas práticas e refatoração de pipeline \(*opcional*\)](#)

Lab 1 — Primeiro pipeline no Azure DevOps

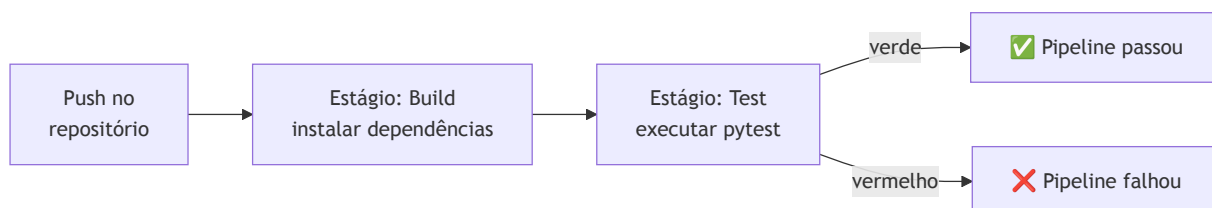
Capítulos relacionados: 2 (Plataformas de CI/CD) e 3 (Anatomia de um Pipeline)

Duração estimada: 40 minutos

Objetivo: criar um repositório no Azure DevOps, adicionar uma aplicação Python com testes unitários e configurar um pipeline YAML que executa build e testes automaticamente a cada push.

O que você vai construir

Um pipeline com dois estágios:



Passo 1 — Criar o repositório no Azure DevOps

1. Acesse **dev.azure.com** e abra o projeto criado no material de apoio
2. No menu lateral, clique em **Repos**
3. Se o repositório estiver vazio, copie a URL HTTPS exibida na tela
4. No terminal da sua máquina, clone o repositório:

```
git clone https://dev.azure.com/SUA-ORGANIZACAO/pipeline-labs/_git/pipeline-labs
cd pipeline-labs
```

Se o repositório não aparecer ou estiver sem URL, clique em **Initialize** para criar o branch inicial.

Passo 2 — Criar a aplicação Python

Dentro da pasta clonada, crie a seguinte estrutura:

```
pipeline-labs/
├─ calculadora.py
├─ test_calculadora.py
└─ requirements.txt
```

calculadora.py

```
def somar(a, b):
    return a + b

def subtrair(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

def dividir(a, b):
    if b == 0:
        raise ValueError("Divisão por zero não é permitida")
    return a / b
```

test_calculadora.py

```
import pytest
from calculadora import somar, subtrair, multiplicar, dividir

def test_somar():
    assert somar(2, 3) == 5
    assert somar(-1, 1) == 0

def test_subtrair():
    assert subtrair(5, 3) == 2

def test_multiplicar():
    assert multiplicar(3, 4) == 12

def test_dividir():
    assert dividir(10, 2) == 5.0

def test_dividir_por_zero():
    with pytest.raises(ValueError):
        dividir(5, 0)
```

requirements.txt

```
pytest==8.2.0
```

Passo 3 — Criar o pipeline YAML

Crie o arquivo `azure-pipelines.yml` na raiz do repositório:

```
trigger:
  - main

pool:
  vmImage: ubuntu-latest

stages:
  - stage: Build
    displayName: "Build — instalar dependências"
    jobs:
      - job: InstalarDependencias
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: "3.11"
              displayName: "Configurar Python 3.11"

          - script: pip install -r requirements.txt
            displayName: "Instalar dependências"

  - stage: Test
    displayName: "Test — executar testes unitários"
    dependsOn: Build
    jobs:
      - job: ExecutarTestes
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: "3.11"
              displayName: "Configurar Python 3.11"

          - script: pip install -r requirements.txt
            displayName: "Instalar dependências"

          - script: pytest test_calculadora.py --tb=short
            displayName: "Executar pytest"
```

Por que instalar dependências nos dois estágios?

Cada estágio roda em um agente limpo e independente. O que foi instalado no estágio Build não persiste automaticamente para o Test. Nas próximas unidades veremos como usar cache para evitar essa repetição.

Passo 4 — Fazer o primeiro push

```
git add .
git commit -m "feat: adiciona calculadora e pipeline CI"
git push origin main
```

Passo 5 — Observar o pipeline no portal

1. No Azure DevOps, acesse **Pipelines**
2. Se for o primeiro pipeline, clique em **Create Pipeline**
 - Selecione **Azure Repos Git**
 - Selecione o repositório **pipeline-labs**
 - Selecione **Existing Azure Pipelines YAML file**
 - Selecione o branch **main** e o arquivo **/azure-pipelines.yml**
 - Clique em **Continue** e depois em **Run**
3. Acompanhe a execução em tempo real — clique em cada estágio para ver os logs

O resultado esperado é:

```
Stage: Build      ✓  
Stage: Test       ✓
```

Passo 6 — Introduzir uma falha intencional

Edite **test_calculadora.py** e altere um valor esperado para forçar uma falha:

```
def test_somar():  
    assert somar(2, 3) == 99 # valor errado propositalmente
```

Faça o push:

```
git add test_calculadora.py  
git commit -m "test: falha intencional para observar pipeline"  
git push origin main
```

Observe que o estágio **Test** falha e o pipeline fica vermelho. Repare que:

- O Azure DevOps envia uma notificação por e-mail sobre a falha
- O log mostra exatamente qual asserção falhou e em qual linha

Passo 7 — Corrigir e fechar o ciclo

Restaure o valor correto:

```
def test_somar():  
    assert somar(2, 3) == 5
```

```
git add test_calculadora.py  
git commit -m "fix: corrige asserção do teste de soma"  
git push origin main
```

O pipeline volta ao verde — ciclo completo de CI funcionando.

Resultado esperado ao final do lab

- Repositório no Azure DevOps com código Python e testes
 - Pipeline YAML com dois estágios (Build e Test) executando automaticamente a cada push
 - Experiência completa do ciclo: push → pipeline verde → falha → correção → verde novamente
-

Pergunta de reflexão

O pipeline atual instala as dependências duas vezes, uma em cada estágio. Em um projeto com dezenas de dependências pesadas, isso dobraria o tempo de execução.

Como você resolveria esse problema sem fundir os dois estágios em um só?

Dica: pesquise sobre *pipeline caching* no Azure DevOps e sobre o conceito de *artifacts* entre estágios.

Lab 2 — Agente auto-hospedado e estratégias de deployment

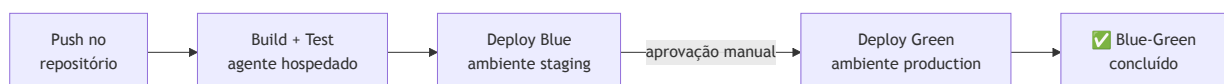
Capítulos relacionados: 2 (Plataformas de CI/CD) e 4 (Estratégias de Deployment)

Duração estimada: 50 minutos

Pré-requisito: Lab 1 concluído. O repositório `pipeline-labs` e o pipeline do Lab 1 devem estar funcionando.

Objetivo: instalar um agente auto-hospedado e evoluir o pipeline para simular duas estratégias de deployment (Rolling Update e Blue-Green) usando os recursos de **Environments** e aprovação manual do Azure DevOps.

O que você vai construir



Passo 1 — Verificar o agente auto-hospedado

Se ainda não instalou o agente, siga os passos do `material-apoio-contas-cicd.md` (seção 3). Ao final, verifique que o agente aparece **Online** em **Organization Settings** → **Agent pools** → **Default**.

Passo 2 — Criar os Environments no Azure DevOps

Os Environments representam ambientes de destino do deploy. Você criará dois: `staging` e `production`.

1. No menu lateral do projeto, acesse **Pipelines** → **Environments**
2. Clique em **New environment**
3. Preencha:
 - **Name:** `staging`
 - **Resource:** None
4. Clique em **Create**
5. Repita para criar o environment `production`

Configurar aprovação manual no `production`:

1. Abra o environment `production`
2. Clique no menu de três pontos (:) no canto superior direito → **Approvals and checks**
3. Clique em + → **Approvals**
4. Em **Approvers**, adicione o seu próprio usuário
5. Clique em **Create**

A partir de agora, qualquer deploy para `production` ficará pausado aguardando sua aprovação antes de prosseguir.

Passo 3 — Adicionar o script de "deploy"

Neste lab o "deploy" é simulado por um script Python que grava um arquivo representando a versão em cada ambiente. Adicione o arquivo `deploy.py` ao repositório:

`deploy.py`

```

import sys
import os
from datetime import datetime

def deploy(ambiente, versao):
    os.makedirs("deploys", exist_ok=True)
    arquivo = f"deploys/{ambiente}.txt"
    conteudo = (
        f"Ambiente: {ambiente}\n"
        f"Versão: {versao}\n"
        f>Data/hora: {datetime.now().isoformat()}\n"
    )
    with open(arquivo, "w") as f:
        f.write(conteudo)
    print(f"[OK] Deploy concluído: {ambiente} → versão {versao}")
    print(conteudo)

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Uso: python deploy.py <ambiente> <versao>")
        sys.exit(1)
    deploy(sys.argv[1], sys.argv[2])

```

Passo 4 — Evoluir o pipeline YAML

Substitua o conteúdo de `azure-pipelines.yml` pelo pipeline completo abaixo:

```

trigger:
  - main

variables:
  VERSAO: "1.0.${Build.BuildId}"

stages:

  # — Estágio 1: Build e Testes
  - stage: BuildTest
    displayName: "Build e Test"
    pool:
      vmImage: ubuntu-latest
    jobs:
      - job: BuildTest
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: "3.11"
              displayName: "Configurar Python 3.11"

          - script: pip install -r requirements.txt
            displayName: "Instalar dependências"

```



```

- script: pytest test_calculadora.py --tb=short
  displayName: "Executar testes"

# — Estágio 2: Deploy Staging (Blue)


---


- stage: DeployStaging
  displayName: "Deploy → Staging (Blue)"
  dependsOn: BuildTest
  jobs:
    - deployment: DeployBlue
      displayName: "Deploy Blue — Staging"
      environment: staging
      pool:
        name: Default # usa o agente auto-hospedado
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self

              - script: python3 deploy.py staging $(VERSAO)
                displayName: "Deploy Blue — Staging"

# — Estágio 3: Deploy Production (Green)


---


- stage: DeployProduction
  displayName: "Deploy → Production (Green)"
  dependsOn: DeployStaging
  jobs:
    - deployment: DeployGreen
      displayName: "Deploy Green — Production"
      environment: production # aprovação manual configurada aqui
      pool:
        name: Default
      strategy:
        runOnce:
          deploy:
            steps:
              - script: python3 deploy.py production $(VERSAO)
                displayName: "Deploy Green — Production"

```

Diferença entre `pool: vmImage` e `pool: name`

- `vmImage: ubuntu-latest` - usa o runner hospedado pela Microsoft (na nuvem, consome minutos do plano gratuito)
- `name: Default` - usa o agente auto-hospedado instalado na sua máquina (sem limite de tempo)

O estágio de Build e Test roda na nuvem; os estágios de deploy rodam no seu agente local.

Passo 5 — Fazer o push das alterações

```
git add azure-pipelines.yml deploy.py
git commit -m "feat: adiciona estratégia Blue-Green com environments"
git push origin main
```

Passo 6 — Observar o pipeline e aprovar o deploy

1. Acompanhe no portal em **Pipelines** → **Runs**
2. O pipeline executará automaticamente os estágios **BuildTest** e **DeployStaging**
3. Ao chegar no estágio **DeployProduction**, ele ficará pausado com o status **Waiting**:
 - Clique no estágio para abrir os detalhes
 - Clique em **Review** → **Approve**
 - O deploy para production prosseguirá

Passo 7 — Verificar o resultado no agente

Ao final, verifique os arquivos gerados pelo script no terminal do agente auto-hospedado.

O Azure DevOps armazena os arquivos em um caminho com um número variável, algo como `~/azagent/_work/<N>/s/`, onde `<N>` é o ID interno do pipeline na sua máquina. Use o comando abaixo para localizar os arquivos independentemente desse número:

```
find ~/azagent/_work -name "staging.txt" -path "**/deploys/*"
2>/dev/null
```

Depois leia os dois arquivos com:

```
cat $(find ~/azagent/_work -name "staging.txt" -path "**/deploys/*"
2>/dev/null)
cat $(find ~/azagent/_work -name "production.txt" -path "**/deploys/*"
2>/dev/null)
```

Dica: se você souber o número que apareceu no seu agente (por exemplo `3`), pode acessar diretamente: `cat ~/azagent/_work/3/s/deploys/staging.txt`

O conteúdo mostrará o ambiente, a versão (com o número do build) e o horário do deploy.

Entendendo o padrão Blue-Green

A ideia central do Blue-Green é manter **dois ambientes idênticos**, um recebe o novo deploy enquanto o outro continua servindo os usuários. A troca é feita apenas após validação.

Neste lab simplificado:

	Blue (Staging)	Green (Production)
Quando recebe o deploy	Automático, após os testes	Somente após aprovação manual
Papel	Validação antes de ir ao ar	Ambiente que serve os usuários
Rollback	Basta não aprovar o deploy Green	Reexecutar o pipeline com versão anterior

Resultado esperado ao final do lab

- Agente auto-hospedado executando os estágios de deploy
- Dois environments configurados no Azure DevOps (*staging* e *production*)
- Aprovação manual bloqueando o avanço para *production*
- Pipeline com três estágios demonstrando o padrão Blue-Green simplificado
- Arquivos *deploys/staging.txt* e *deploys/production.txt* com versão e horário de cada deploy

Pergunta de reflexão

No Blue-Green real, o rollback consiste em redirecionar o tráfego de volta para o ambiente anterior sem um novo deploy. O pipeline que você construiu não tem essa capacidade, ele precisaria de um novo push para reverter.

O que seria necessário adicionar ao pipeline para suportar um rollback sem novo push?

Dica: pense em um estágio de *rollback* condicional acionado manualmente, e em como a variável `$(VERSAO)` poderia ser substituída por um parâmetro informado pelo operador no momento da execução.

Lab 3 — GitOps com Azure DevOps e repositório de manifests

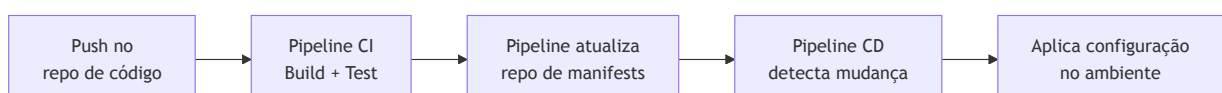
Capítulo relacionado: 5 (GitOps)

Duração estimada: 45 minutos

Pré-requisito: Labs 1 e 2 concluídos.

Objetivo: separar o repositório de código do repositório de configuração (manifests), implementando o padrão GitOps onde o pipeline de CI atualiza o repositório de configuração e um pipeline separado aplica as mudanças no ambiente.

O que você vai construir



Este é o padrão **Push-based GitOps**: o pipeline de CI empurra a mudança para o repositório de configuração, que dispara automaticamente o pipeline de CD. O ambiente nunca é alterado diretamente pelo pipeline de CI, toda mudança passa pelo repositório de manifests.

Passo 1 — Criar o repositório de manifests

Você terá dois repositórios separados dentro do mesmo projeto no Azure DevOps:

Repositório	Papel
<code>pipeline-labs</code>	Código-fonte da aplicação (já existe)
<code>pipeline-manifests</code>	Arquivos de configuração do ambiente

Para criar o segundo repositório:

1. No Azure DevOps, acesse **Repos**
2. Clique no nome do repositório atual (canto superior) → **New repository**
3. Preencha:
 - **Repository name:** `pipeline-manifests`
 - **Add a README:** marcado
4. Clique em **Create**
5. Clone o novo repositório na sua máquina:

```
git clone https://dev.azure.com/SUA-ORGANIZACAO/pipeline-labs/_git/pipeline-manifests
cd pipeline-manifests
```

Passo 2 — Criar o manifest de configuração inicial

Dentro do repositório `pipeline-manifests`, crie a seguinte estrutura:

```
pipeline-manifests/
├─ environments/
│   ├── staging.json
│   └─ production.json
```

environments/staging.json

```
{
  "ambiente": "staging",
  "versao": "1.0.0",
  "atualizado_em": "2025-01-01T00:00:00"
}
```

environments/production.json

```
{
  "ambiente": "production",
  "versao": "1.0.0",
  "atualizado_em": "2025-01-01T00:00:00"
}
```

Faça o commit inicial:

```
git add .
git commit -m "feat: manifests iniciais de staging e production"
git push origin main
```

Passo 3 — Criar um Personal Access Token para o pipeline

O pipeline de CI precisará de permissão para fazer push no repositório `pipeline-manifests`. Crie um PAT dedicado:

1. No Azure DevOps, clique no ícone de usuário → **Personal access tokens**
2. Clique em **New Token**
3. Preencha:
 - **Name:** `pipeline-gitops`
 - **Expiration:** 90 dias
 - **Scopes** → **Code:** marque **Read & write**
4. Clique em **Create** e copie o token

Adicionar o token como variável secreta no pipeline:

1. No Azure DevOps, acesse **Pipelines** e abra o pipeline do `pipeline-labs`
2. Clique em **Edit** → **Variables** (ícone no canto superior direito)
3. Clique em **New variable**
4. Preencha:
 - **Name:** `MANIFESTS_PAT`
 - **Value:** cole o token
 - Marque **Keep this value secret**
5. Clique em **OK** e salve

Passo 4 — Criar o script de atualização de manifest

No repositório `pipeline-labs`, adicione o arquivo `atualizar_manifest.py`:

atualizar_manifest.py

```

import sys
import json
import subprocess
from datetime import datetime

def atualizar_manifest(ambiente, versao, repo_url):
    # Clonar o repositório de manifests
    subprocess.run(
        ["git", "clone", repo_url, "manifests-repo"],
        check=True
    )

    # Atualizar o arquivo JSON do ambiente
    caminho = f"manifests-repo/environments/{ambiente}.json"
    manifest = {
        "ambiente": ambiente,
        "versao": versao,
        "atualizado_em": datetime.now().isoformat()
    }
    with open(caminho, "w") as f:
        json.dump(manifest, f, indent=2, ensure_ascii=False)
        f.write("\n")

    # Commit e push
    subprocess.run(["git", "-C", "manifests-repo", "config",
        "user.email", "pipeline@devops"], check=True)
    subprocess.run(["git", "-C", "manifests-repo", "config",
        "user.name", "Pipeline CI"], check=True)
    subprocess.run(["git", "-C", "manifests-repo", "add", "."],
        check=True)
    subprocess.run(
        ["git", "-C", "manifests-repo", "commit", "-m",
            f"ci: atualiza {ambiente} para versão {versao}"],
        check=True
    )
    subprocess.run(["git", "-C", "manifests-repo", "push"],
        check=True)
    print(f"[OK] Manifest de {ambiente} atualizado para versão {versao}")

if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Uso: python atualizar_manifest.py <ambiente> <versao> <repo_url>")
        sys.exit(1)
    atualizar_manifest(sys.argv[1], sys.argv[2], sys.argv[3])

```

Passo 5 — Criar o pipeline de CD no repositório de manifests

No repositório `pipeline-manifests`, crie o arquivo `azure-pipelines.yml`:

`pipeline-manifests/azure-pipelines.yml`

```

trigger:
  - main

pool:
  name: Default    # agente auto-hospedado

steps:
  - checkout: self

  - script: |
      echo "=== Aplicando configuração de staging ==="
      cat environments/staging.json

      echo ""
      echo "=== Aplicando configuração de production ==="
      cat environments/production.json

      echo ""
      echo "[OK] Configurações aplicadas com sucesso"
      displayName: "Aplicar manifests no ambiente"

```

Em um cenário real com Kubernetes, os comandos `cat` seriam substituídos por `kubectl apply -f` apontando para os arquivos de deployment. Neste lab, exibir o conteúdo do manifest representa a etapa de aplicação.

Faça o commit:

```

git add azure-pipelines.yml
git commit -m "feat: adiciona pipeline CD ao repositório de manifests"
git push origin main

```

Registre o pipeline no Azure DevOps apontando para este repositório (mesmo processo do Lab 1, Passo 5).

Passo 6 — Evoluir o pipeline de CI para atualizar o manifest

No repositório `pipeline-labs`, substitua o conteúdo de `azure-pipelines.yml`:

```

trigger:
  - main

variables:
  VERSAO: "1.0.${Build.BuildId}"
  MANIFESTS_REPO: "https://$(MANIFESTS_PAT)@dev.azure.com/SUA-ORGANIZACAO/pipeline-labs/_git/pipeline-manifests"

stages:
  # Etapa 1: Build o Manifest

```

— Estágio 1: Build e testes

```
- stage: BuildTest
  displayName: "Build e Test"
  pool:
    vmImage: ubuntu-latest
  jobs:
    - job: BuildTest
      steps:
        - task: UsePythonVersion@0
          inputs:
            versionSpec: "3.11"
            displayName: "Configurar Python 3.11"

        - script: pip install -r requirements.txt
          displayName: "Instalar dependências"

        - script: pytest test_calculadora.py --tb=short
          displayName: "Executar testes"
```

— Estágio 2: Atualizar manifest de staging

```
- stage: UpdateStagingManifest
  displayName: "GitOps → Atualizar manifest de staging"
  dependsOn: BuildTest
  pool:
    name: Default
  jobs:
    - job: AtualizarStaging
      steps:
        - checkout: self

        - script: >
            python3 atualizar_manifest.py
            staging
            $(VERSAO)
            $(MANIFESTS_REPO)
          displayName: "Atualizar manifest de staging"
```

— Estágio 3: Atualizar manifest de production (com aprovação)

```
- stage: UpdateProductionManifest
  displayName: "GitOps → Atualizar manifest de production"
  dependsOn: UpdateStagingManifest
  pool:
    name: Default
  jobs:
    - deployment: AtualizarProduction
      environment: production
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self

              - script: >
```



```
python3 atualizar_manifest.py
production
$(VERSAO)
$(MANIFESTS_REPO)
displayName: "Atualizar manifest de production"
```

Lembre de substituir `SUA-ORGANIZACAO` pelo nome real da sua organização no Azure DevOps.

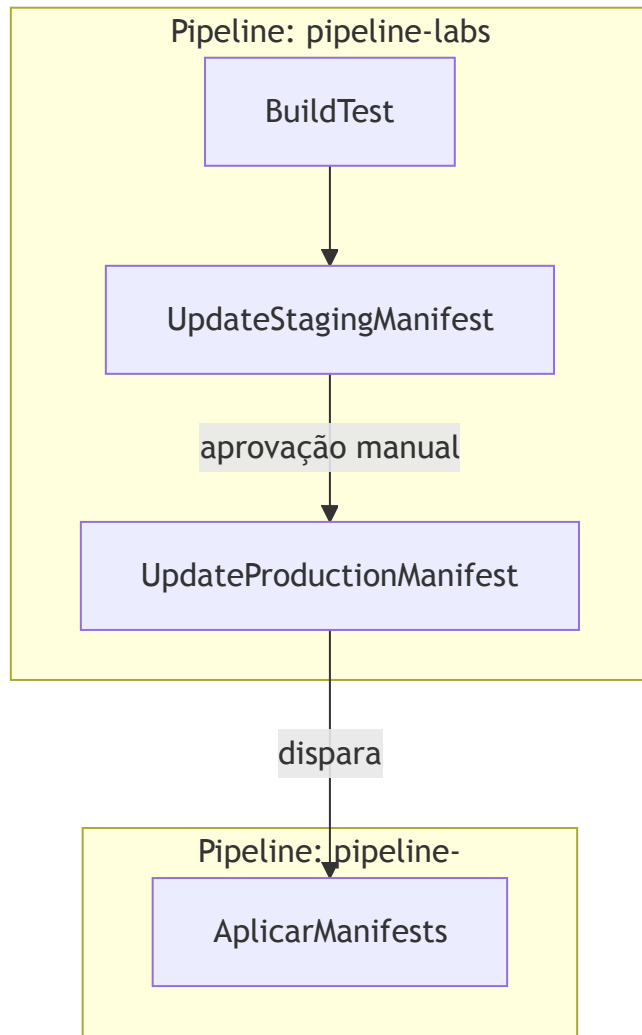
Passo 7 — Executar o fluxo GitOps completo

1. Faça o push das alterações no repositório `pipeline-labs`:

```
git add azure-pipelines.yml atualizar_manifest.py
git commit -m "feat: pipeline CI atualiza repositório de manifests"
git push origin main
```

2. Acompanhe o pipeline de CI em **Pipelines** → **pipeline-labs**
3. Após aprovação, o estágio `UpdateProductionManifest` faz push no `pipeline-manifests`
4. O push no `pipeline-manifests` dispara automaticamente o pipeline de CD desse repositório
5. Acompanhe o segundo pipeline em **Pipelines** → **pipeline-manifests**

O fluxo completo no portal ficará assim:



Resultado esperado ao final do lab

- Dois repositórios separados: código em `pipeline-labs` e configuração em `pipeline-manifests`
- Pipeline de CI atualiza os arquivos JSON dos ambientes após cada push aprovado
- Pipeline de CD é acionado automaticamente pela mudança no repositório de manifests
- O ambiente nunca é alterado diretamente pelo CI, toda mudança passa pelo repositório de configuração

Pergunta de reflexão

Neste lab o pipeline de CI faz **push** no repositório de manifests, que então dispara o CD — isso é o padrão **Push-based GitOps**. No padrão **Pull-based** (usado pelo ArgoCD e Flux), é o próprio ambiente que fica observando o repositório e se auto-atualiza.

Qual das duas abordagens oferece mais segurança em um ambiente corporativo, e por quê?

Dica: pense em quem precisa ter credenciais de acesso ao ambiente de produção em cada um dos modelos.

Lab 4 — Boas práticas e refatoração de pipeline (opcional)

Capítulo relacionado: 6 (Boas Práticas de Design e Manutenção de Pipelines)

Duração estimada: 40 minutos

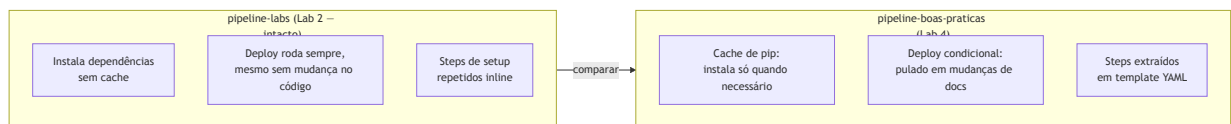
Pré-requisito: Labs 1 e 2 concluídos.

Objetivo: criar um novo repositório com o mesmo código do Lab 2 e aplicar três melhorias concretas no pipeline: cache de dependências, execução condicional e templates reutilizáveis. Ao final, os dois repositórios ficam disponíveis lado a lado para comparação.

Este lab é opcional. Ele não introduz novos conceitos de CI/CD, mas consolida boas práticas que fazem diferença em projetos reais.

O que você vai construir

Você partirá de uma cópia do pipeline do Lab 2 em um repositório novo — sem alterar o `pipeline-labs` original. Ao final, será possível comparar as duas versões diretamente no portal:



Passo 1 — Criar o novo repositório

No Azure DevOps, crie um segundo repositório dentro do mesmo projeto:

1. Acesse **Repos** no menu lateral
2. Clique no nome do repositório atual (canto superior esquerdo) para abrir o seletor
3. Clique em **New repository**
4. Preencha:
 - **Repository name:** `pipeline-boas-praticas`
 - **Add a README:** marcado (se desmarcado, a branch default criada será `master` ao invés de `main`).
5. Clique em **Create**

Clone o novo repositório na sua máquina (a URL está em **Repos** → **Clone**):

```
git clone <URL-do-pipeline-boas-praticas>
cd pipeline-boas-praticas
```

Passo 2 — Criar os arquivos de aplicação

O Lab 3 altera o repositório `pipeline-labs` para o padrão GitOps, então não é seguro copiar os arquivos de lá. Crie-os diretamente no novo repositório com o conteúdo do Lab 2:

`calculadora.py`

```
def somar(a, b):  
    return a + b  
  
def subtrair(a, b):  
    return a - b  
  
def multiplicar(a, b):  
    return a * b  
  
def dividir(a, b):  
    if b == 0:  
        raise ValueError("Divisão por zero não é permitida")  
    return a / b
```

test_calculadora.py

```
import pytest  
from calculadora import somar, subtrair, multiplicar, dividir  
  
def test_somar():  
    assert somar(2, 3) == 5  
    assert somar(-1, 1) == 0  
  
def test_subtrair():  
    assert subtrair(5, 3) == 2  
  
def test_multiplicar():  
    assert multiplicar(3, 4) == 12  
  
def test_dividir():  
    assert dividir(10, 2) == 5.0  
  
def test_dividir_por_zero():  
    with pytest.raises(ValueError):  
        dividir(5, 0)
```

requirements.txt

```
pytest==8.2.0
```

deploy.py

```

import sys
import os
from datetime import datetime

def deploy(ambiente, versao):
    os.makedirs("deploys", exist_ok=True)
    arquivo = f"deploys/{ambiente}.txt"
    conteudo = (
        f"Ambiente: {ambiente}\n"
        f"Versão: {versao}\n"
        f>Data/hora: {datetime.now().isoformat()}\n"
    )
    with open(arquivo, "w") as f:
        f.write(conteudo)
    print(f"[OK] Deploy concluído: {ambiente} → versão {versao}")
    print(conteudo)

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Uso: python deploy.py <ambiente> <versao>")
        sys.exit(1)
    deploy(sys.argv[1], sys.argv[2])

```

Crie o `azure-pipelines.yml` inicial — este é o ponto de partida, idêntico ao resultado final do Lab 2:

azure-pipelines.yml

```

trigger:
  - main

variables:
  VERSAO: "1.0.$(Build.BuildId)"

stages:

  # — Estágio 1: Build e Test
  

---


  - stage: BuildTest
    displayName: "Build e Test"
    pool:
      vmImage: ubuntu-latest
    jobs:
      - job: BuildTest
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: "3.11"
              displayName: "Configurar Python 3.11"

          - script: pip install -r requirements.txt
            displayName: "Instalar dependências"

```

```

- script: pytest test_calculadora.py --tb=short
  displayName: "Executar testes"

# — Estágio 2: Deploy Staging (Blue)


---


- stage: DeployStaging
  displayName: "Deploy → Staging (Blue)"
  dependsOn: BuildTest
  jobs:
    - deployment: DeployBlue
      displayName: "Deploy Blue – Staging"
      environment: staging
      pool:
        name: Default
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self

              - script: python3 deploy.py staging $(VERSAO)
                displayName: "Deploy Blue – Staging"

# — Estágio 3: Deploy Production (Green)


---


- stage: DeployProduction
  displayName: "Deploy → Production (Green)"
  dependsOn: DeployStaging
  jobs:
    - deployment: DeployGreen
      displayName: "Deploy Green – Production"
      environment: production
      pool:
        name: Default
      strategy:
        runOnce:
          deploy:
            steps:
              - script: python3 deploy.py production $(VERSAO)
                displayName: "Deploy Green – Production"

```

Faça o primeiro push:

```

git add .
git commit -m "chore: cópia inicial do pipeline do Lab 2"
git push origin main

```

Passo 3 — Conectar o repositório a um pipeline no Azure DevOps

1. Acesse **Pipelines** → **New pipeline**

2. Selecione **Azure Repos Git** e escolha `pipeline-boas-praticas`
3. Selecione **Existing Azure Pipelines YAML file** e aponte para `/azure-pipelines.yml`
4. Clique em **Run** para confirmar que o pipeline base funciona antes de aplicar as melhorias

Os environments `staging` e `production` já existem do Lab 2 — não é necessário recriá-los. Porém, como este é um **pipeline novo**, o Azure DevOps exige que você autorize o acesso a cada recurso na primeira execução.

Autorizações necessárias na primeira execução:

Ao chegar no estágio **Deploy** → **Staging (Blue)**, o pipeline pausará com a mensagem:

"This pipeline needs permission to access 2 resources before this run can continue to Deploy → Staging (Blue)"

Clique em **View** e autorize os dois recursos:

- **Environment: staging**
- **Agent pool: Default**

O estágio de Staging prosseguirá. Ao chegar no estágio **Deploy** → **Production (Green)**, o pipeline pausará novamente com:

"This pipeline needs permission to access a resource before this run can continue to Deploy → Production (Green)"

Clique em **View** e autorize:

- **Environment: production**

Após essa autorização, aparecerá a aprovação manual normal do environment `production`. Clique em **Review** → **Approve** para liberar o deploy para produção.

Essas autorizações são solicitadas apenas uma vez por pipeline. Nas execuções seguintes, os recursos já estarão liberados e o fluxo seguirá normalmente.

Passo 4 — Adicionar cache de dependências

O pipeline atual baixa os pacotes do pip em toda execução. Com cache, o download ocorre apenas quando o `requirements.txt` muda.

Edite o `azure-pipelines.yml`: adicione a variável `PIP_CACHE` e substitua os steps de setup no estágio `BuildTest`:

```

variables:
  VERSAO: "1.0.${Build.BuildId}"
  PIP_CACHE: $(Pipeline.Workspace)/.pip # ← nova variável

stages:

  # — Estágio 1: Build e Test
  -----

  - stage: BuildTest
    displayName: "Build e Test"
    pool:
      vmImage: ubuntu-latest
    jobs:
      - job: BuildTest
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: "3.11"
              displayName: "Configurar Python 3.11"

          - task: Cache@2 # ← novo step
            inputs:
              key: 'pip | "$(Agent.OS)" | requirements.txt'
              restoreKeys: |
                pip | "$(Agent.OS)"
              path: $(PIP_CACHE)
              displayName: "Cache de dependências pip"

          - script: pip install --cache-dir $(PIP_CACHE) -r
            requirements.txt
            displayName: "Instalar dependências" # ← agora usa o
            cache

          - script: pytest test_calculadora.py --tb=short
            displayName: "Executar testes"

```

Como o cache funciona:

- Na primeira execução, o pip baixa os pacotes e os salva em `$(PIP_CACHE)`
- O Azure DevOps armazena esse diretório com a chave `pip | OS | requirements.txt`
- Nas execuções seguintes, se o `requirements.txt` não mudou, o cache é restaurado e o download é pulado
- Se o `requirements.txt` mudar, a chave muda e o cache é invalidado automaticamente

```

git add azure-pipelines.yml
git commit -m "perf: adiciona cache de dependências pip"
git push origin main

```


Passo 5 — Adicionar execução condicional

O deploy não precisa rodar quando a mudança foi apenas em documentação ou testes. Adicione um filtro de caminhos no trigger e uma condição explícita nos estágios de deploy.

Substitua o bloco `trigger` e adicione `condition` nos dois estágios de deploy:

```
trigger:
  branches:
    include:
      - main
  paths:
    exclude:
      - "*.md"
      - "**/*.md"
      - "test_*.py"
```

Atenção: no Azure DevOps, `**/*.md` cobre arquivos `.md` em subpastas, mas **não** os da raiz do repositório. Por isso é necessário incluir `*.md` separadamente para cobrir a raiz.

— Estágio 2: Deploy Staging (Blue)

```
- stage: DeployStaging
  displayName: "Deploy → Staging (Blue)"
  dependsOn: BuildTest
  condition: and(succeeded(), ne(variables['Build.Reason'],
'PullRequest'))    # ← novo
```

— Estágio 3: Deploy Production (Green)

```
- stage: DeployProduction
  displayName: "Deploy → Production (Green)"
  dependsOn: DeployStaging
  condition: and(succeeded(), ne(variables['Build.Reason'],
'PullRequest'))    # ← novo
```

O que essas condições fazem:

- `paths.exclude` — commits que alteram apenas `.md` ou `test_*.py` não disparam o pipeline
- `succeeded()` — o estágio só executa se o anterior passou
- `ne(...'PullRequest')` — o deploy não roda em builds de Pull Request, apenas em merges diretos para `main`

Faça o push das alterações do pipeline antes de validar — o `paths.exclude` só entra em vigor após o `azure-pipelines.yml` atualizado estar no repositório:

```
git add azure-pipelines.yml
git commit -m "ci: execução condicional por paths e por tipo de build"
git push origin main
```

Após esse push confirmar que o pipeline roda normalmente, valide o comportamento alterando apenas o README:

```
echo "# pipeline-boas-praticas" > README.md
git add README.md
git commit -m "docs: atualiza README"
git push origin main
```

O pipeline **não deve ser disparado** desta vez, o `paths.exclude` bloqueia commits que afetam apenas `.md`. Verifique em **Pipelines** → **Runs** que nenhuma nova execução foi criada.

Passo 6 — Extrair steps de setup em um template

Os três steps de configuração do Python (versão, cache, instalação) ficam inline no `BuildTest`. Extraia-os para um template para que qualquer mudança futura como trocar a versão do Python, por exemplo, seja feita em um único lugar.

Crie a pasta e o arquivo de template:

templates/setup-python.yml

```
steps:
- task: UsePythonVersion@0
  inputs:
    versionSpec: "3.11"
    displayName: "Configurar Python 3.11"

- task: Cache@2
  inputs:
    key: 'pip | "$(Agent.OS)" | requirements.txt'
    restoreKeys: |
      pip | "$(Agent.OS)"
    path: $(PIP_CACHE)
    displayName: "Cache de dependências pip"

- script: pip install --cache-dir $(PIP_CACHE) -r requirements.txt
  displayName: "Instalar dependências"
```

Substitua os três steps inline no `BuildTest` pela referência ao template:

```

- stage: BuildTest
  displayName: "Build e Test"
  pool:
    vmImage: ubuntu-latest
  jobs:
    - job: BuildTest
      steps:
        - template: templates/setup-python.yml  # ← substitui os
três steps

        - script: pytest test_calculadora.py --tb=short
          displayName: "Executar testes"

```

Faça o push com as duas mudanças:

```

git add azure-pipelines.yml templates/
git commit -m "refactor: extrai setup de Python em template
reutilizável"
git push origin main

```

Estado final dos arquivos

Ao final do lab, o repositório `pipeline-boas-praticas` terá esta estrutura:

```

pipeline-boas-praticas/
├── azure-pipelines.yml
├── templates/
│   └── setup-python.yml
├── calculadora.py
├── test_calculadora.py
├── deploy.py
└── requirements.txt

```

azure-pipelines.yml completo:

```

trigger:
  branches:
    include:
      - main
  paths:
    exclude:
      - "*.md"
      - "**/*.md"
      - "test_*.py"

variables:
  VERSAO: "1.0.$(Build.BuildId)"
  PIP_CACHE: $(Pipeline.Workspace)/.pip

```

stages:

— Estágio 1: Build e Test

```
- stage: BuildTest
  displayName: "Build e Test"
  pool:
    vmImage: ubuntu-latest
  jobs:
    - job: BuildTest
      steps:
        - template: templates/setup-python.yml

        - script: pytest test_calculadora.py --tb=short
          displayName: "Executar testes"
```

— Estágio 2: Deploy Staging (Blue)

```
- stage: DeployStaging
  displayName: "Deploy → Staging (Blue)"
  dependsOn: BuildTest
  condition: and(succeeded(), ne(variables['Build.Reason'],
'PullRequest'))
  jobs:
    - deployment: DeployBlue
      displayName: "Deploy Blue - Staging"
      environment: staging
      pool:
        name: Default
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: self

              - script: python3 deploy.py staging $(VERSAO)
                displayName: "Deploy Blue - Staging"
```

— Estágio 3: Deploy Production (Green)

```
- stage: DeployProduction
  displayName: "Deploy → Production (Green)"
  dependsOn: DeployStaging
  condition: and(succeeded(), ne(variables['Build.Reason'],
'PullRequest'))
  jobs:
    - deployment: DeployGreen
      displayName: "Deploy Green - Production"
      environment: production
      pool:
        name: Default
      strategy:
        runOnce:
          deploy:
            steps:
```

```
- script: python3 deploy.py production $(VERSAO)
  displayName: "Deploy Green - Production"
```

Resultado esperado ao final do lab

- Repositório `pipeline-boas-praticas` criado e funcionando em paralelo ao `pipeline-labs` original
- Cache de pip configurado, reduzindo o tempo de instalação nas execuções subsequentes
- Deploy condicional: commits em `.md` e arquivos de teste não disparam os estágios de deploy
- Template YAML extraído, eliminando os steps de setup inline

Comparando os dois pipelines no portal

Com os dois repositórios ativos, acesse **Pipelines** e abra execuções de cada um lado a lado:

	<code>pipeline-labs</code> (Lab 2)	<code>pipeline-boas-praticas</code> (Lab 4)
Tempo de "Instalar dependências"	Igual em todas as execuções	Reduz nas execuções com cache
Push de <code>.md</code>	Dispara o pipeline inteiro	Pipeline não é disparado
Manutenção da versão do Python	Editar em cada estágio	Editar apenas em <code>templates/setup-python.yml</code>

Pergunta de reflexão

O template criado neste lab vive dentro do mesmo repositório. Em times com múltiplos repositórios e múltiplos pipelines, cada equipe teria que copiar e manter seu próprio template.

Como você centralizaria os templates para que todos os times de um mesmo projeto pudessem reutilizá-los sem duplicação?

Dica: pesquise sobre *template repositories* no Azure DevOps e o recurso `repository` dentro do bloco `resources` de um pipeline YAML.